NOVA SCHOOL OF SCIENCE & TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE

MEI - Master in Computer Science and Engineering

Internet Applications Design and Implementation

Week 6 - Security in SSR Applications. Spring Security.

Information

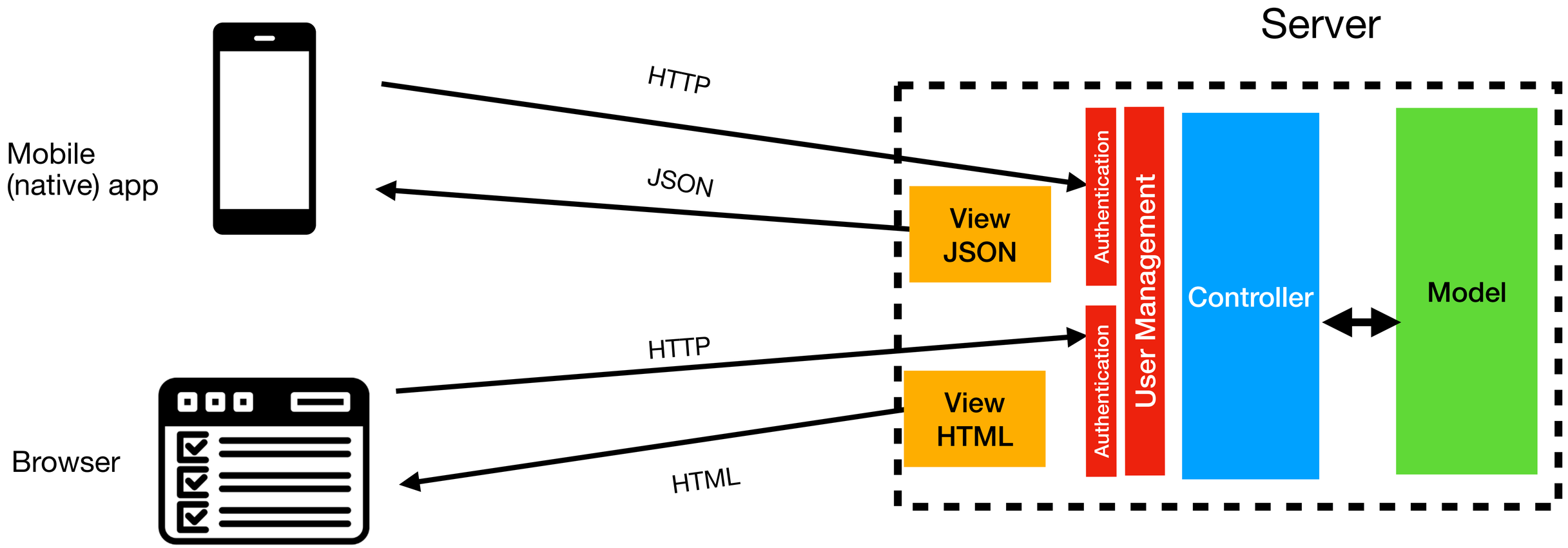
Project (Part 1) submissions included in the practical-grades document available in CLIP (Documents > Other)

This was mandatory (although non-graded)

Next week, the instructions for Part 2 will be released

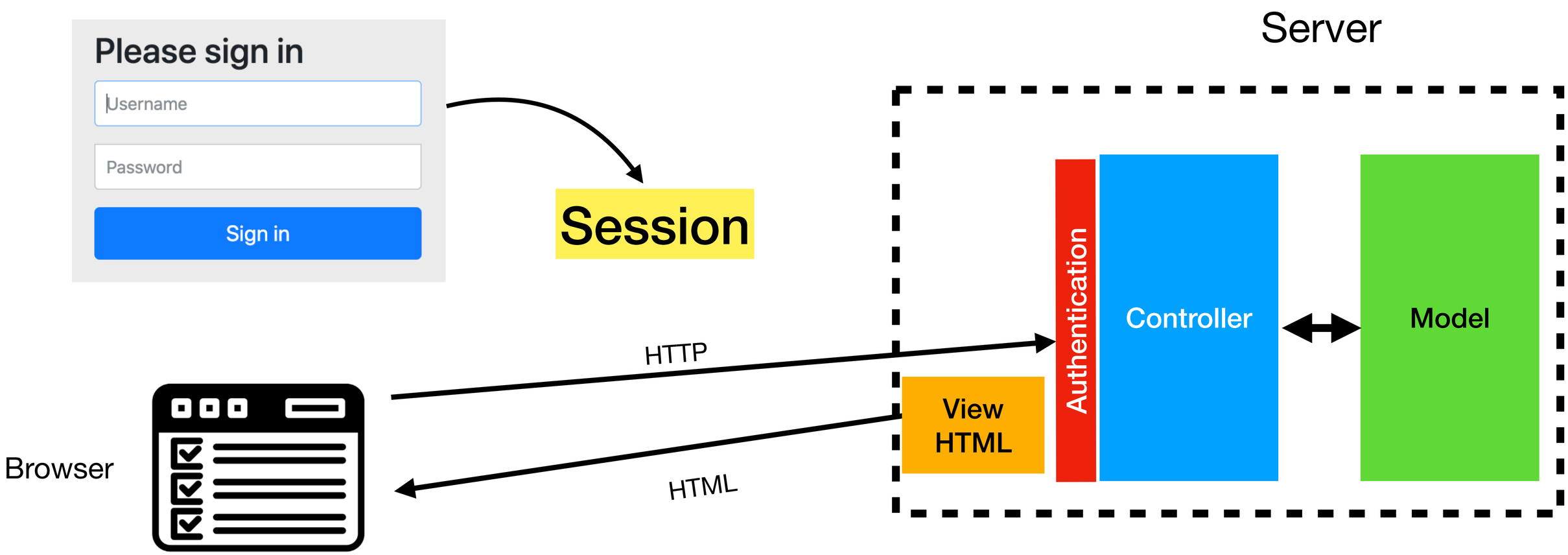
© Pedro Alves 2026

Authentication in Internet Applications



© Pedro Alves 2026

Authentication in Internet Applications

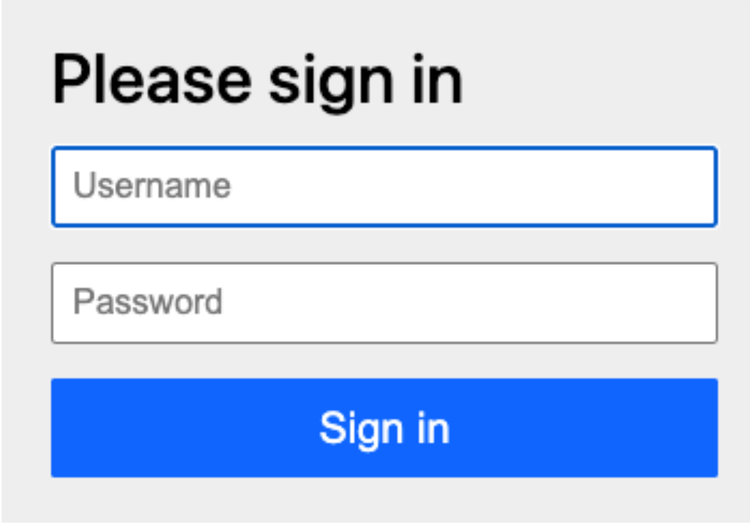


© Pedro Alves 2026

Spring Security

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

Only by adding this dependency, your entire SSR application is now protected behind a login



This is managed by Spring, you don't need to create this html page neither the controller that handles it

© Pedro Alves 2026

Spring Security

A default user (named “user”) is created with a random password printed to the console (different every time)

```
2026-03-27T14:11:04.981Z  WARN 17895 --- [app]
[       main] .s.s.UserDetailsServiceAutoConfiguration :

Using generated security password: 3d403564-4f84-4ecf-ba65-343583c0ff24

This generated password is for development use only. Your security configuration must be updated
before running your application in production.
```

© Pedro Alves 2026

Spring Security

Securing an SSR application with HTTP Basic Authentication is similar to RESTful

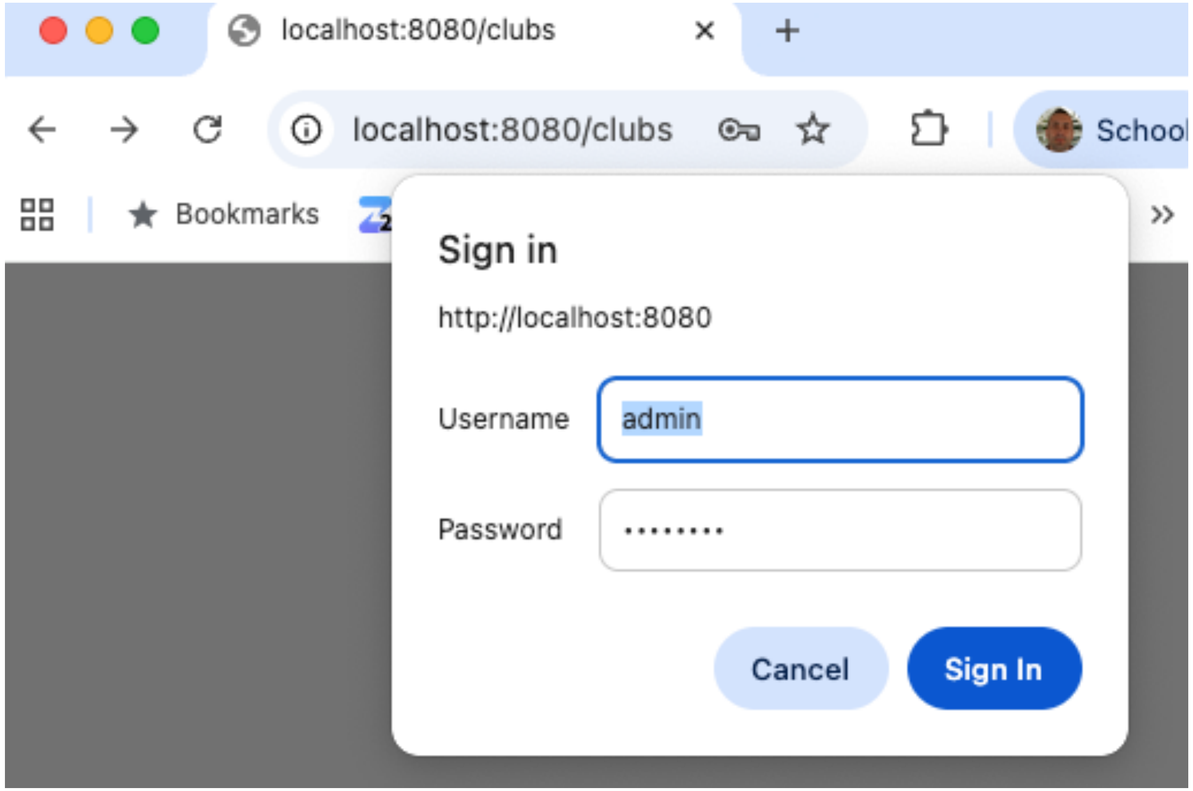
```
@Configuration
@EnableWebSecurity
class SecurityConfig {

    @Bean
    fun passwordEncoder(): PasswordEncoder =
                                BCryptPasswordEncoder()

    @Bean
    fun userDetailsService(encoder: PasswordEncoder) {
       …
    }

    @Bean
    fun securityFilterChain(http: HttpSecurity): SecurityFilterChain {
        http
            .csrf { it.disable() }
            .authorizeHttpRequests { auth ->
                auth
                    .requestMatchers(HttpMethod.GET, "/posts", "/posts/**").permitAll()
                    .requestMatchers(HttpMethod.POST, "/posts").hasAnyRole("EDITOR", "ADMIN")
                    .requestMatchers(HttpMethod.PUT, "/posts/**").hasAnyRole("EDITOR", "ADMIN")
                    .requestMatchers(HttpMethod.DELETE, "/posts/**").hasRole("ADMIN")
                    .anyRequest().authenticated()
            }
            .httpBasic {}

        return http.build()
    }
}
```

remove this line to enable CSRF protection

© Pedro Alves 2026

HTTP Basic Auth

Browsers automatically handle HTTP Basic Auth by showing a modal dialog for entering credentials



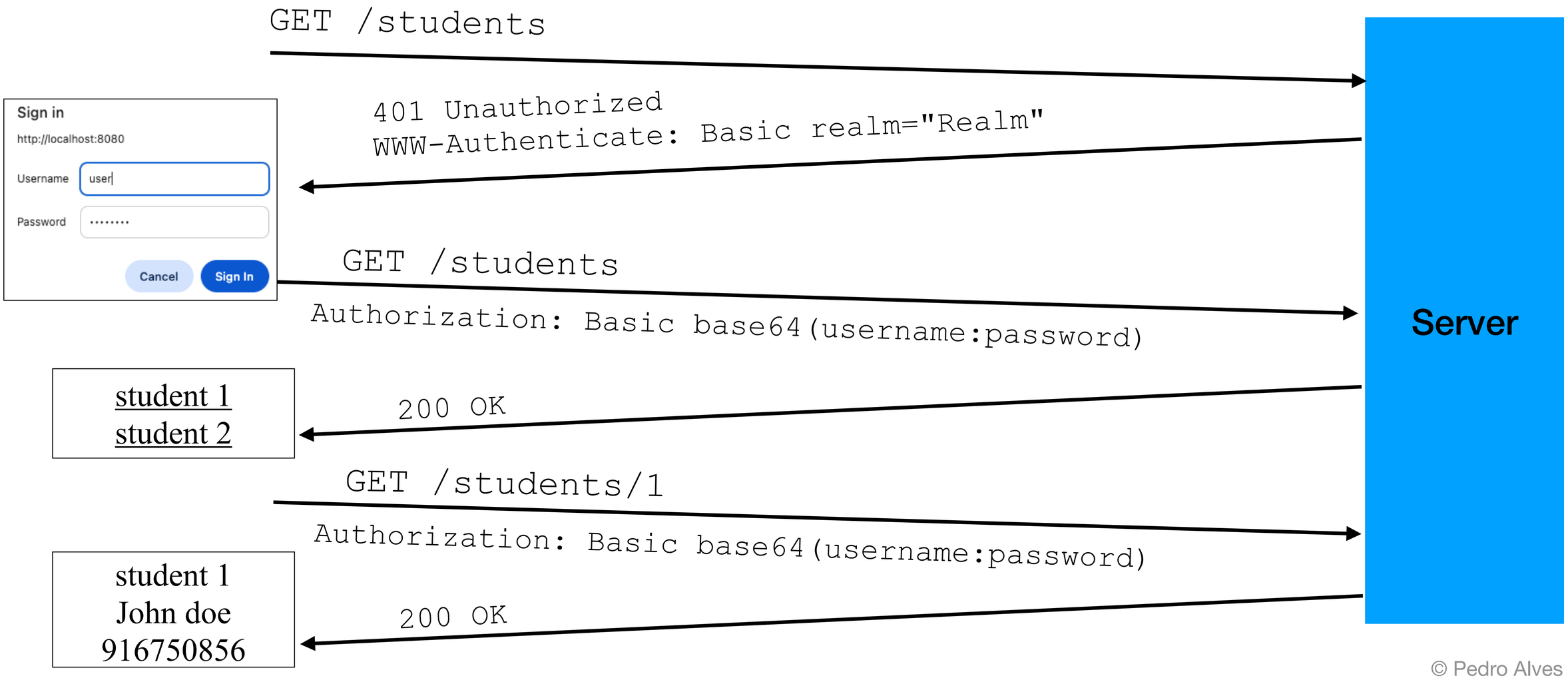
The credentials are kept in memory until you leave the site and sent again in every request

No session needed

© Pedro Alves 2026

Authentication in Internet Applications

HTTP Basic Auth



© Pedro Alves 2026

HTTP Basic Auth - Spring

```
@Configuration
@EnableWebSecurity
class SecurityConfig {

    @Bean
    fun securityFilterChain(http: HttpSecurity): SecurityFilterChain {

        http
            .authorizeHttpRequests { auth ->
                auth.anyRequest().authenticated()
            }
            .httpBasic {
                it.realmName("My App") // optional, default is "Realm"
            }

        return http.build()
    }
}
```

© Pedro Alves 2026

HTTP Basic Auth - Spring

Benefits

- Simple to implement
- No session management

Drawbacks

- Credentials sent on every request - higher risk than sending tokens or other temporary information
- No “logout” mechanism - User must close the browser
- No control over the UI of the login form

© Pedro Alves 2026

Authentication in Internet Applications

Form-based Login

```
GET /students
                    302 Redirect
                    Location: /login
GET /login
                    200 OK
POST /login
Payload: username, password, _csrf
                    302 Redirect
                    Location: /students; Set-Cookie: JSESSIONID=123
GET /students   Cookie: JSESSIONID=123
                    200 OK
```

Server

student 1
student 2

© Pedro Alves 2026

Spring

Form-based Login

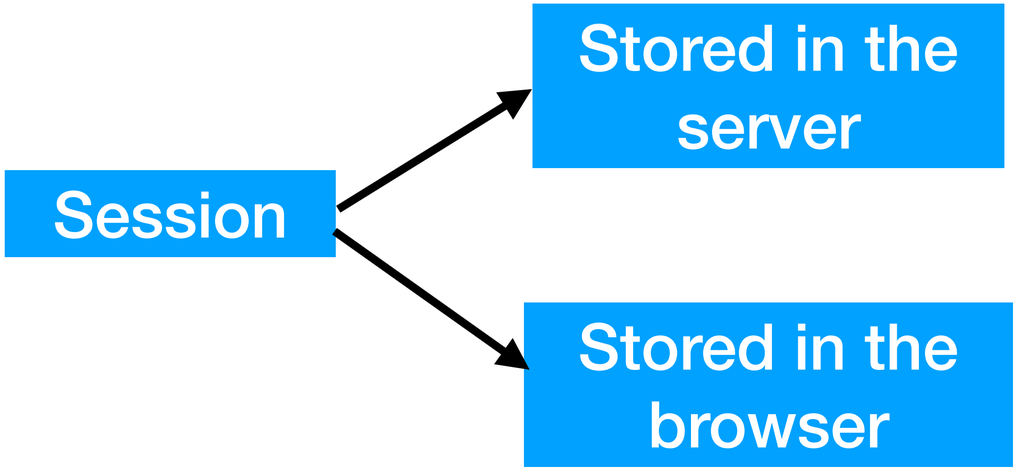
```
@Configuration
@EnableWebSecurity
class SecurityConfig {

    @Bean
    fun securityFilterChain(http: HttpSecurity): SecurityFilterChain {

        http
            .authorizeHttpRequests { auth ->
                auth.anyRequest().authenticated()
            }
            .formLogin { }

        return http.build()
    }
}
```

© Pedro Alves 2026

Session

HTTP is stateless

Sessions allow websites to temporarily store and retrieve user-specific state, enabling continuity across multiple requests



Browser sends a session id on every request
The server lookups the information associated with that id (in memory, db, etc…)

Browser sends the whole session information on every request

© Pedro Alves 2026

Session stored in the server

Sessions are assigned unique IDs per browser

Information stored “in the session” is actually usually stored in memory, in a Map-like structure (key=sessionId)

session id (key)	session data (value)
35D43A656	{ “username”: “Pedro”, … }
14FJ007C1	{ “username”: “Cris”, … }
7D90A90D4	{ “username”: “Sara”, … }

HTTP Requests include the session ID

The server lookups the session data associated with that ID and provides it to the controller/service layer

© Pedro Alves 2026

Session stored in the server

HTTP Requests include the session ID:

- Cookies
- URL Rewriting
- Hidden fields

© Pedro Alves 2026

Cookies

- Small pieces of data stored in the user's browser
- Sent automatically with every HTTP request <u>to the same domain</u>

How it works:

1. Server responds with a Set-Cookie header
2. Browser stores the cookie
3. Browser sends it back in future requests (Cookie header)
4. Server uses it (e.g., to identify a session)

© Pedro Alves 2026

Cookies

Common uses:

- Session management (login state)
- User preferences (language, theme)
- Tracking & analytics

```
Set-Cookie: JSESSIONID=abc123; HttpOnly; Secure
```

Security considerations:

- HttpOnly - not accessible via JavaScript (prevents XSS theft)
- Secure - sent only over HTTPS
- SameSite - protects against CSRF attacks

© Pedro Alves 2026

Cookies

Lifetime:

- **Session Cookies** - Until the browser is closed (default)
- **Persistent Cookies** - Defined by the server (Max-Age / Expires attributes)

```
Set-Cookie: theme=dark; Expires=Wed, 01 Jan 2027 00:00:00 GMT
```

The logout operation deletes only session cookies

© Pedro Alves 2026

Spring

Cookies

Use @CookieValue to read cookies

```
@Controller
@RequestMapping("/students")
class StudentsController {

    @GetMapping
    fun list(@CookieValue(name = "JSESSIONID", required = false)
             sessionId: String?): String {

        println("sessionId = ${sessionId}")
        ...
    }
```

© Pedro Alves 2026

Spring

Cookies

Use ResponseCookie and HttpServletResponse to write cookies

```
@GetMapping("/login")
fun login(response: HttpServletResponse) {
    val cookie = ResponseCookie.from("sessionId", "abc123")
        .httpOnly(true)
        .secure(true)
        .path("/")
        .maxAge(3600)
        .sameSite("Strict")
        .build()

    response.addHeader("Set-Cookie", cookie.toString())
}
```

© Pedro Alves 2026

Spring

Cookies

There is no operation to delete cookies

On logout, you must clear and force the expiration of the cookies

```
Set-Cookie: JSESSIONID=; Max-Age=0; Expires=Thu, 01 Jan 1970 00:00:10 GMT;
```

© Pedro Alves 2026

URL Rewriting

Session ID is appended to URLs

```
/dashboard;jsessionid=ABC123
```

- Used when cookies are disabled
- Session ID exposed in URLs (security risk)
- Can leak via logs, bookmarks, or sharing
- In Spring, it can be enabled by setting this flag in application.properties: `server.servlet.session.tracking-modes=url`

© Pedro Alves 2026

Hidden Form Fields

Session ID stored in hidden <input> fields inside forms

Sent only when forms are submitted

Not suitable for general navigation

```
<form action="/transfer" method="post">

    <input type="hidden" name="sessionId" value="abc123xyz" />

    <label>Amount:</label>
    <input type="text" name="amount" />

    <label>Recipient:</label>
    <input type="text" name="to" />

    <button type="submit">Send</button>
</form>
```

© Pedro Alves 2026

Session stored in the browser

If you have many active sessions, your server's memory may run out

Alternative: store the session in browser

- Cookies with whole session data
- sessionStorage
- localStorage
- IndexedDB

require javascript; more suited for CSR applications

© Pedro Alves 2026

Cookies

You can store as much as 4 KB of info per cookie

For example, a JWT fits in this limit

```
{
  "header": {
    "alg": "HS256",
    "typ": "JWT"
  },
  "payload": {
    "sub": "1234567890",
    "name": "John Doe",
    "role": "admin",
    "scope": "read:orders write:orders",
    "iat": 1516239022
  },
  "signature": "SflKxwRJSMeKKF2QT4fw..."
}
```

⚠️ Beware of increased network bandwidth consumption

The whole cookie is sent on every request

© Pedro Alves 2026

JWT

JSON Web Token

- A compact, URL-safe token format used for authentication & authorization
- Commonly used in web apps and APIs
- Encoded as a JSON object
- Can be encrypted and/or digitally signed
- Defined by the standard: RFC 7519 JSON Web Token

© Pedro Alves 2026

JWT

- **Header**
  - Algorithm (e.g., HS256, RS256)
  - Token type (JWT)
- **Payload**
  - Contains user data, roles, expiration, etc..
- **Signature**
  - Verifies integrity using a secret or private key
  - `HMACSHA256(base64Url(header) + "." + base64Url(payload),secret)`

```json
{
  "header": {
    "alg": "HS256",
    "typ": "JWT"
  },
  "payload": {
    "sub": "1234567890",
    "name": "John Doe",
    "role": "admin",
    "scope": "read:orders write:orders",
    "iat": 1516239022
  },
  "signature": "SflKxwRJSMeKKF2QT4fw..."
}
```

Represented as:

`base64Url(header).base64Url(payload).base64Url(signature)`

Example: `eyJhbGcIkpXVCJ9fh1.eyJ1c2Vy6ImFkbWluIn0.SflKxwRJSMeKKPOk6yJV_adQssw5c`

© Pedro Alves 2026

Spring
JWT

Java JWT: JSON Web Token for Java and Android

```
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.13.0</version>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-impl</artifactId>
    <version>0.13.0</version>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>0.13.0</version>
    <scope>runtime</scope>
</dependency>
```

API and Implementation for parsing and creating JWT tokens

© Pedro Alves 2026

Spring
JWT

application.properties

```
jwt.secret=novaevents-jwt-secret-key-for-hs256-signing
```

```
@Service
class JwtService(@Value("\${jwt.secret}") secret: String) {

    private val key: SecretKey = Keys.hmacShaKeyFor(secret.toByteArray())
    private val expirationMs = 3_600_000L // 1 hour

    fun generate(name: String, roles: List<String>): String =
        Jwts.builder()
            .subject(name)
            .claim("roles", roles)
            .issuedAt(Date())
            .expiration(Date(System.currentTimeMillis() + expirationMs))
            .signWith(key)
            .compact()
}
```

© Pedro Alves 2026

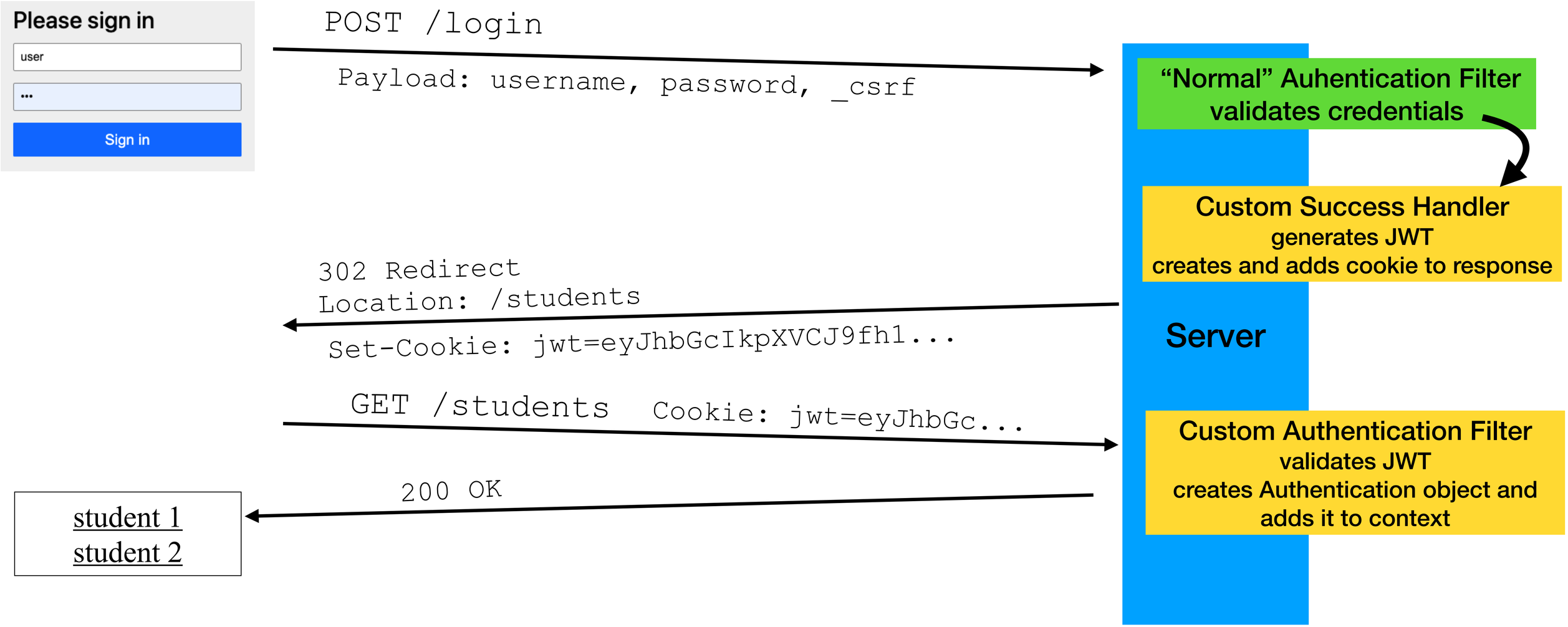
Spring
JWT

```
@Service
class JwtService(@Value("\${jwt.secret}") secret: String) {

    private val key: SecretKey = Keys.hmacShaKeyFor(secret.toByteArray())
    private val expirationMs = 3_600_000L // 1 hour

    …

    fun validate(token: String): Map<String, Any>? =
        runCatching {
            val claims: Claims = Jwts.parser()
                .verifyWith(key)
                .build()
                .parseSignedClaims(token)
                .payload
            @Suppress("UNCHECKED_CAST")
            mapOf("name" to claims.subject, "roles" to claims["roles"] as List<String>)
        }.getOrNull()
}
```

© Pedro Alves 2026

Spring

Cookies with JWT



© Pedro Alves 2026

Spring

Cookies with JWT

```
@Bean
fun securityFilterChain(http: HttpSecurity): SecurityFilterChain {
    http
        .sessionManagement {
            it.sessionCreationPolicy(SessionCreationPolicy.STATELESS)
        }

        .addFilterBefore(jwtCookieAuthFilter,
                         UsernamePasswordAuthenticationFilter::class.java)

        .authorizeHttpRequests {   ...   }
        .formLogin { form ->
            form.loginPage("/login").permitAll()
            form.successHandler(jwtAuthSuccessHandler)
        }
   …
}
```

Don't use "normal" JSESSIONID sessions

Set our custom authentication filter

Set our custom success handler

© Pedro Alves 2026

Spring

Cookies with JWT

```
@Component
class JwtAuthSuccessHandler(private val jwtService: JwtService) : AuthenticationSuccessHandler {

    override fun onAuthenticationSuccess(
        request: HttpServletRequest,
        response: HttpServletResponse,
        authentication: Authentication
    ) {
        val token = jwtService.generate(authentication.name,
                                        authentication.authorities.map { it.authority })
        val cookie = Cookie("jwt", token).apply {
            isHttpOnly = true
            path = "/"
            maxAge = 3600
        }
        response.addCookie(cookie)
        response.sendRedirect(request.contextPath + "/")
    }
}
```

This was filled in by the “normal” login filter

Generates JWT token

Creates the cookie, httpOnly (to prevent js access)

Appends the cookie to the response headers (Set-Cookie: …)

Redirect to “/“, doesn’t handle ongoing login, more on that later…

© Pedro Alves 2026

Spring

Cookies with JWT

```
@Component
class JwtCookieAuthFilter(private val jwtService: JwtService) : OncePerRequestFilter() {

    private val securityContextRepository = RequestAttributeSecurityContextRepository()

    override fun doFilterInternal(request: HttpServletRequest, response: HttpServletResponse, filterChain: FilterChain) {
        if (SecurityContextHolder.getContext().authentication == null) {
            val token = request.cookies?.find { it.name == "jwt" }?.value
            if (token != null) {
                val claims = jwtService.validate(token)
                if (claims != null) {
                    @Suppress("UNCHECKED_CAST")
                    val authorities = (claims["roles"] as List<String>).map { SimpleGrantedAuthority(it) }
                    val context = SecurityContextHolder.createEmptyContext()
                    context.authentication =
                        UsernamePasswordAuthenticationToken(claims["name"], null, authorities)
                    SecurityContextHolder.setContext(context)
                    securityContextRepository.saveContext(context, request, response)
                }
            }
        }
        filterChain.doFilter(request, response)
    }
}
```

gets the JWT from the cookie

validates the JWT

fills in the Authentication object

saves in the context

© Pedro Alves 2026

Spring

Cookies with JWT

Application
- Manifest
- Service workers
- Storage

Storage
- Local storage
- Session storage
- Extension storage
- IndexedDB
- Cookies
  - http://localhost:8080
- Private state tokens

Filter | Only show cookies with an issu

Name	Value	Do...	Path	Ex...	Size	Htt...	Se
csrftoken	RkJRx9IEF88Hjmhqc0mf4E8...	loc...	/	20...	41		
jwt	eyJhbGciOiJIUzI1NiJ9.eyJzd...	loc...	/	20...	166	✓	
XSRF-TOKEN	b9348eef-6ba8-410c-9a33...	loc...	/	Se...	46		

Get up-to-speed with JSON Web Tokens. Get the JWT Handbook for free

JWT Debugger — Debugger | Introduction | Libraries | Ask

Paste a JWT below that you'd like to decode, validate, and verify.

Encoded Token — Enable auto-focus

JSON Web Token (JWT)

eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhbGljZSIsInJvbGVzIjpbIlJPTEVfRURJVE9SIl0sImlhdCI6MTc3NDk1NDU5NywiZXhwIjoxNzc0OTU4MTk3fQ.0hnh5eBzoO7n7_sqZzm92il-6REcWsV0NjfASm1e8RQ

Decoded Header — JSON | Claims Breakdown

```
{
  "alg": "HS256"
}
```

Decoded Payload — JSON | Claims Breakdown

```
{
  "sub": "alice",
  "roles": [
    "ROLE_EDITOR"
  ]
```

No JSESSIONID!

https://jwt.io

© Pedro Alves 2026

Authentication in Internet Applications

Session

Exercise

	Cookie (Session ID)	URL Rewriting	Hidden Form Field	Cookie (JWT)	Web Storage (local/session/IDB)
State Location (browser, server)					
Automatic transmission (yes/no)					
Transmission Scope (All requests, Forms Only, JS-Controlled)					
Server memory consumption with session info (high, low, none)					
Most fit model (SSR, CSR)					

© Pedro Alves 2026

Authentication in Internet Applications

Session

	Cookie (Session ID)	URL Rewriting	Hidden Form Field	Cookie (JWT)	Web Storage (local/session/IDB)
State Location (browser, server)	server	server	server	browser	browser
Automatic transmission (yes/no)	yes	no	no	yes	no
Transmission Scope (All requests, Forms Only, JS-Controlled)	all	all	forms-only	all	js-controlled
Server memory consumption with session info (high, low, none)	high	high	high	none	none
Most fit model (SSR, CSR)	SSR	SSR	SSR	Both	CSR

© Pedro Alves 2026

Spring

Session Management

Spring controls how a session is created and how Spring Security will handle it

- **always**: a session will always be created if one doesn't already exist
- **ifRequired**: a session will be created only if required (default)
- **never**: the framework will never create a session itself but it will use one if it already exists
- **stateless**: no session will be created or used by Spring Security

```
http
    .sessionManagement {
        it.sessionCreationPolicy(SessionCreationPolicy.IF_REQUIRED)
    }
```

© Pedro Alves 2026

Spring

Working with sessions

Spring injects an HttpSession object with methods for reading and writing to the session

```
@GetMapping("/set")
fun setSession(session: HttpSession): String {
    session.setAttribute("username", "alice")
    return "stored"
}

@GetMapping("/get")
fun getSession(session: HttpSession): String {
    val username = session.getAttribute("username") as? String
    return "User: $username"
}
```

© Pedro Alves 2026

Spring

Working with sessions

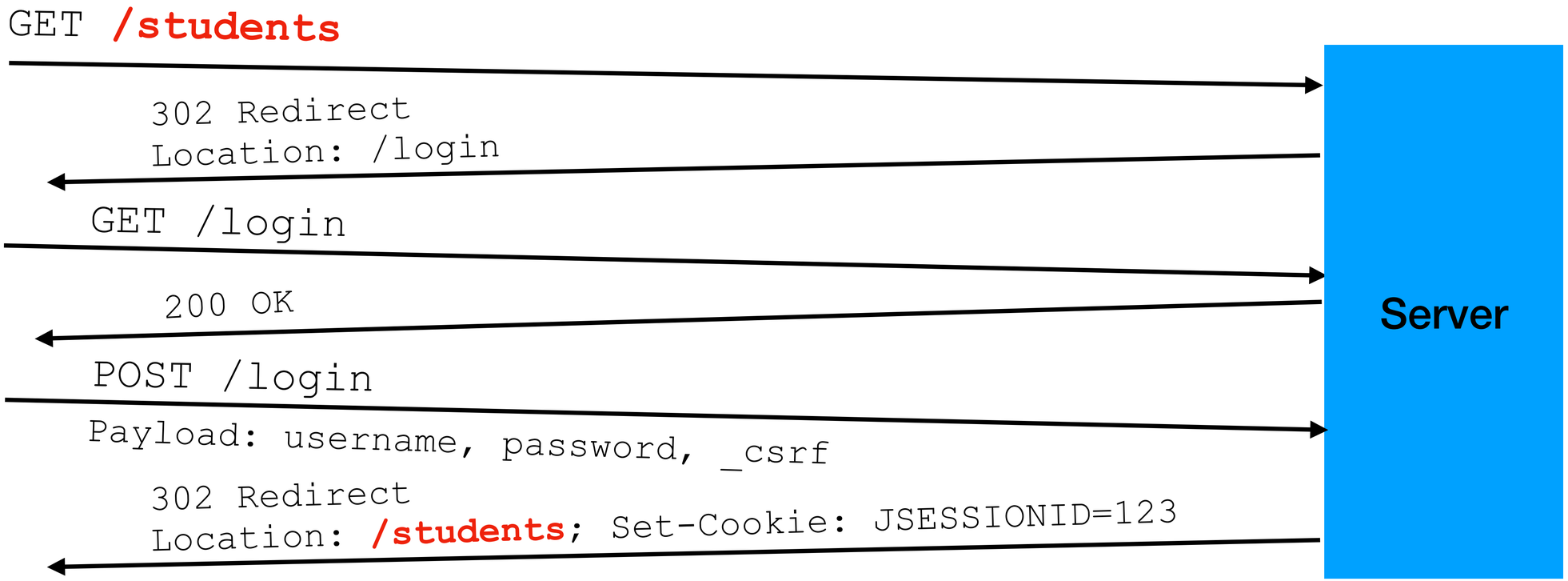
Another option is to declare a bean with scope “session”.
This bean is created when the session is first created and linked to the lifecycle of the HTTPSession object

```
@Component
@Scope("session", proxyMode = ScopedProxyMode.TARGET_CLASS)
class SessionInfo(var numberOfGets:Int = 0, var numberOfPosts:Int = 0)

@Controller
class StudentController(private val sessionInfo: SessionInfo) {

    @GetMapping
    fun list(model: ModelMap): String {
        sessionInfo.numberOfGets++
        println("Number of gets: ${sessionInfo.numberOfGets}")
        ...
    }
```

© Pedro Alves 2026

Spring

Ongoing Login



How does it know where to redirect to?

© Pedro Alves 2026

Spring

Ongoing Login

GET /students

302 Redirect
Location: /login ;Set-Cookie: JSESSIONID=123

Filter stores URL in RequestCache (HttpSession)

GET /login

200 OK

Server

POST /login

Payload: username, password, _csrf
Cookie: JSESSIONID=123

Filter gets previous URL from RequestCache

302 Redirect
Location: /students

© Pedro Alves 2026

Spring

Customizing Form-based Login

```
@Bean
fun securityFilterChain(http: HttpSecurity): SecurityFilterChain {

    http
        .authorizeHttpRequests { auth ->
            auth.anyRequest().authenticated()
        }
        .formLogin {
            it.loginPage("/login").permitAll()
        }

    return http.build()
}
```

The login form page is now our responsiblity

Controller

```
@GetMapping("/login")
fun showLogin() = "login/login"
```

login/login.html (thymeleaf)

```
<form th:action="@{/login}" method="post">
   <input type="text" id="username" name="username"/>
   <input type="password" id="password" name="password"/>

   <button type="submit">Sign In</button>
</form>
```

Note: thymeleaf automatically includes the _csrf hidden field

© Pedro Alves 2026

Spring

Customizing Form-based Login

`POST /login`

Automatically handled by Spring Security (UsernamePasswordAuthenticationFilter)
Assumes username and password form fields
Can be overriden by the properties loginProcessingUrl, usernameParameter, passwordParameter

`POST /logout`

Automatically handled by Spring Security (LogoutFilter)
Can be overriden by the property logoutUrl

© Pedro Alves 2026

Spring

Stateless Session Management

```
http
    .sessionManagement {
        it.sessionCreationPolicy(SessionCreationPolicy.STATELESS)
    }
    ...
```

To reduce server memory usage and improve scalability, stateless applications do not use HTTP sessions, using JWT in a cookie instead.

Several built-in features will stop working because they rely on sessions:

- CSRF token validation
- Ongoing login

© Pedro Alves 2026

Stateless Session Management

CSRF Token validation - session-enabled

On session-enabled applications, when you access a form:

- A CSRF token is generated and stored in the session (server memory)
- The form page is rendered with a hidden field (_csrf) containing the token
  thymeleaf does this automatically
- Upon POSTing that form, the _csrf parameter is checked against the value in the session

© Pedro Alves 2026

Stateless Session Management

CSRF Token validation - session-disabled

On session-disabled applications, when you access any URL:

- A new CSRF token is generated and stored in a cookie (XSRF-TOKEN)
  The previous XSRF-TOKEN is cleared/expired
- The form page is rendered with a hidden field (_csrf) containing the token
  thymeleaf does this automatically
- Upon POSTing that form, the _csrf parameter is checked against the value in the XSRF-TOKEN cookie

© Pedro Alves 2026

Spring

Stateless Session Management

CSRF Token validation - session-disabled

```
http
    .sessionManagement {
        it.sessionCreationPolicy(SessionCreationPolicy.STATELESS)
    }
    .securityContext {
        it.securityContextRepository(RequestAttributeSecurityContextRepository())
    }
    .csrf { csrf ->
        csrf.csrfTokenRepository(CookieCsrfTokenRepository())
        csrf.csrfTokenRequestHandler(CsrfTokenRequestAttributeHandler())
    }
    ...
```

Use these settings, otherwise the cookie-based CSRF validation won’t work

© Pedro Alves 2026

Spring

Stateless Session Management

Ongoing Login

1. Access a protected URL
2. Redirect to /login
3. POST credentials
4. Upon success, redirect back to the original protected URL

© Pedro Alves 2026

Stateless Session Management

Ongoing Login - session-enabled

On session-enabled applications, when you access a page the requires authentication:

- Before redirecting to “/login”, Spring stores the original URL/params/headers in the session (`HttpSessionRequestCache`)
- After successful authentication, Spring redirects to the URL stored in the session (`SavedRequestAwareAuthenticationSuccessHandler`)

© Pedro Alves 2026

Stateless Session Management

Ongoing Login - session-disabled

On session-disabled applications, when you access a page the requires authentication:

- Before redirecting to “/login”, Spring stores the original URL/params/headers in a cookie “REDIRECT_URI” (through `CookieRequestCache`)
- After successful authentication, Spring redirects to the URL stored in the cookie (handled by a custom `AuthenticationSuccessHandler`)

© Pedro Alves 2026

Spring

Stateless Session Management

Ongoing Login - session-disabled

```
@Bean
fun securityFilterChain(http: HttpSecurity): SecurityFilterChain {
    http
        ...
        .requestCache { it.requestCache(CookieRequestCache()) }
        ...
```

© Pedro Alves 2026

Spring

Stateless Session Management

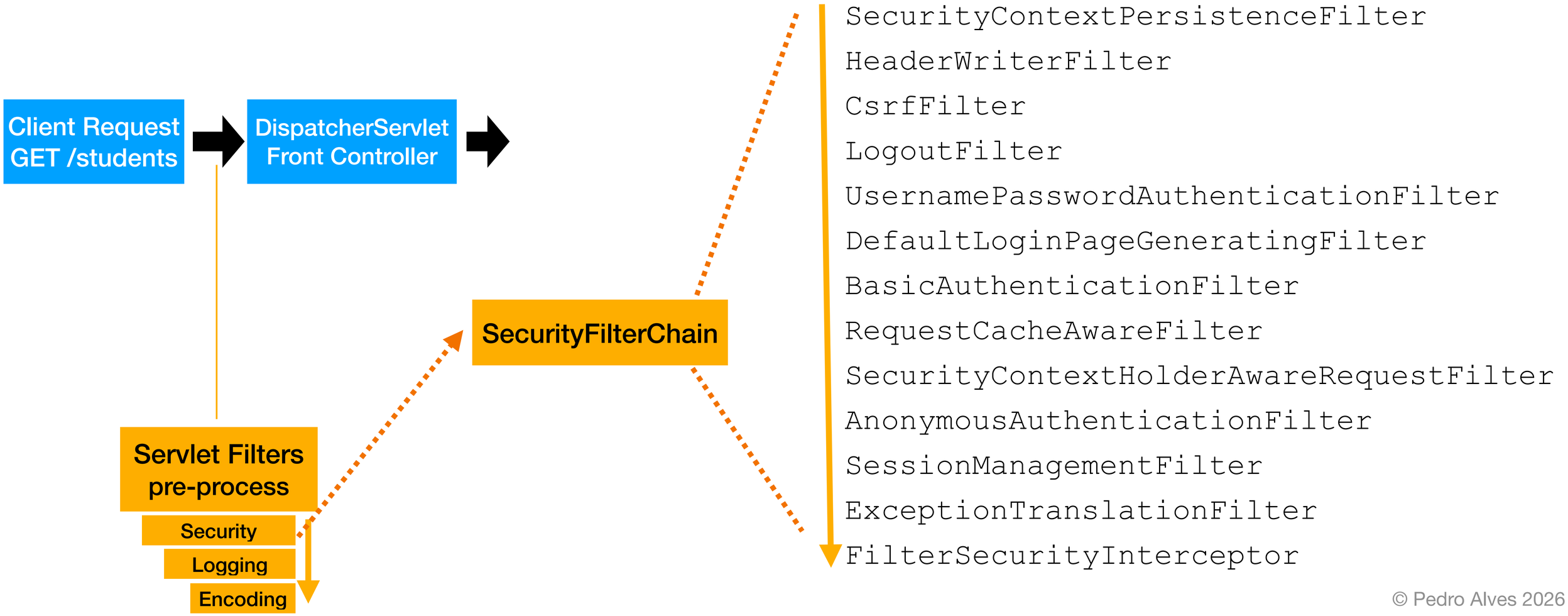
Ongoing Login - session-disabled

```
@Component
class JwtAuthSuccessHandler(private val jwtService: JwtService) : AuthenticationSuccessHandler {

    private val requestCache = CookieRequestCache()

    override fun onAuthenticationSuccess(request: HttpServletRequest,response: HttpServletResponse,
                                         authentication: Authentication) {
        val token = jwtService.generate(authentication.name, authentication.authorities.map { it.authority })
        val cookie = Cookie("jwt", token).apply {
            isHttpOnly = true
            path = "/"
            maxAge = 3600
        }
        response.addCookie(cookie)
        val savedRequest = requestCache.getRequest(request, response)
        val redirectUrl = savedRequest?.redirectUrl ?: (request.contextPath + "/")
        requestCache.removeRequest(request, response)
        response.sendRedirect(redirectUrl)
    }
}
```

© Pedro Alves 2026

Spring

SecurityFilterChain



© Pedro Alves 2026

Spring

SecurityFilterChain

```
SecurityContextPersistenceFilter
HeaderWriterFilter
CsrfFilter
LogoutFilter
UsernamePasswordAuthenticationFilter
DefaultLoginPageGeneratingFilter
BasicAuthenticationFilter
RequestCacheAwareFilter
SecurityContextHolderAwareRequestFilter
AnonymousAuthenticationFilter
SessionManagementFilter
ExceptionTranslationFilter
FilterSecurityInterceptor
```

Enabled by default

Prevents CSRF attacks:

- generating a token and storing it into the session or a cookie (through the `CsrfTokenRepository`)
- validating the `_csrf` parameter that must be sent on all non GET requests

© Pedro Alves 2026

Spring

SecurityFilterChain

SecurityContextPersistenceFilter
HeaderWriterFilter
CsrfFilter
LogoutFilter
UsernamePasswordAuthenticationFilter
DefaultLoginPageGeneratingFilter
BasicAuthenticationFilter
RequestCacheAwareFilter
SecurityContextHolderAwareRequestFilter
AnonymousAuthenticationFilter
SessionManagementFilter
ExceptionTranslationFilter
FilterSecurityInterceptor

Only acts when the request matches the logout url (usually POST /logout)

Clears the authentication object and invalidates session-related state

Cookie is removed using `Set-Cookie: JSESSIONID=; Max-Age=0`

Not automatic on session stateless configs

```
http
    .logout { logout ->
        logout.deleteCookies("jwt")
        ...
    }
    ...
```

© Pedro Alves 2026

Spring

SecurityFilterChain

```
SecurityContextPersistenceFilter
HeaderWriterFilter
CsrfFilter
LogoutFilter
UsernamePasswordAuthenticationFilter
DefaultLoginPageGeneratingFilter
BasicAuthenticationFilter
RequestCacheAwareFilter
SecurityContextHolderAwareRequestFilter
AnonymousAuthenticationFilter
SessionManagementFilter
ExceptionTranslationFilter
FilterSecurityInterceptor
```

Only acts when the request matches the login url (usually POST /login)
(that's why you don't need to implement a login controller)

- Extracts username and password
- Hands them (indirectly) to the UserDetailsService
- On success stores the resulting Authentication in the SecurityContext
- On failure it triggers the configured authentication failure handling (usually redirect to /login?error)

© Pedro Alves 2026

Spring

SecurityFilterChain

```
SecurityContextPersistenceFilter
HeaderWriterFilter
CsrfFilter
LogoutFilter
UsernamePasswordAuthenticationFilter
DefaultLoginPageGeneratingFilter
BasicAuthenticationFilter
RequestCacheAwareFilter
SecurityContextHolderAwareRequestFilter
AnonymousAuthenticationFilter
SessionManagementFilter
ExceptionTranslationFilter
FilterSecurityInterceptor
```

Only acts if you enabled form login and did not provide your own login page

Generates and returns a default login page

© Pedro Alves 2026

Spring

SecurityFilterChain

```
SecurityContextPersistenceFilter
HeaderWriterFilter
CsrfFilter
LogoutFilter
UsernamePasswordAuthenticationFilter
DefaultLoginPageGeneratingFilter
BasicAuthenticationFilter
RequestCacheAwareFilter
SecurityContextHolderAwareRequestFilter
AnonymousAuthenticationFilter
SessionManagementFilter
ExceptionTranslationFilter
FilterSecurityInterceptor
```

Checks for an `Authorization: Basic ...` header

If the header is present, it decodes credentials, attempts authentication, and, on success, places the authenticated principal into the SecurityContext

© Pedro Alves 2026

Spring

SecurityFilterChain

```
SecurityContextPersistenceFilter
HeaderWriterFilter
CsrfFilter
LogoutFilter
UsernamePasswordAuthenticationFilter
DefaultLoginPageGeneratingFilter
BasicAuthenticationFilter
RequestCacheAwareFilter
SecurityContextHolderAwareRequestFilter
AnonymousAuthenticationFilter
SessionManagementFilter
ExceptionTranslationFilter
FilterSecurityInterceptor
```

Handles “ongoing login”, redirecting the user back to where he was before login

© Pedro Alves 2026

Spring

SecurityFilterChain

```
SecurityContextPersistenceFilter
HeaderWriterFilter
CsrfFilter
LogoutFilter
UsernamePasswordAuthenticationFilter
DefaultLoginPageGeneratingFilter
BasicAuthenticationFilter
RequestCacheAwareFilter
SecurityContextHolderAwareRequestFilter
AnonymousAuthenticationFilter
SessionManagementFilter
ExceptionTranslationFilter
FilterSecurityInterceptor
```

Handles **authentication** (e.g., redirecting to the login page or returning a 401)

and **authorization** errors (e.g., redirecting to an error page or returning a 403)

© Pedro Alves 2026

Spring

SecurityFilterChain

```
SecurityContextPersistenceFilter
HeaderWriterFilter
CsrfFilter
LogoutFilter
UsernamePasswordAuthenticationFilter
DefaultLoginPageGeneratingFilter
BasicAuthenticationFilter
RequestCacheAwareFilter
SecurityContextHolderAwareRequestFilter
AnonymousAuthenticationFilter
SessionManagementFilter
ExceptionTranslationFilter
FilterSecurityInterceptor
```

Applies authorization rules

© Pedro Alves 2026

Spring

SecurityFilterChain

Are these filters in the right order?

```
...
ExceptionTranslationFilter
FilterSecurityInterceptor
```

handles **authorization** errors (e.g., redirecting to an error page or returning a 403)

Applies authorization rules

© Pedro Alves 2026

Spring

SecurityFilterChain

The SecurityFilterChain follows the decorated intercepting filter pattern

ExceptionTranslationFilter

```
try {
    chain.doFilter(request, response);
} catch (AccessDeniedException e) {
    // return 403 or delegate
}
```

FilterSecurityInterceptor

checks authorization rules, throws AccessDeniedException

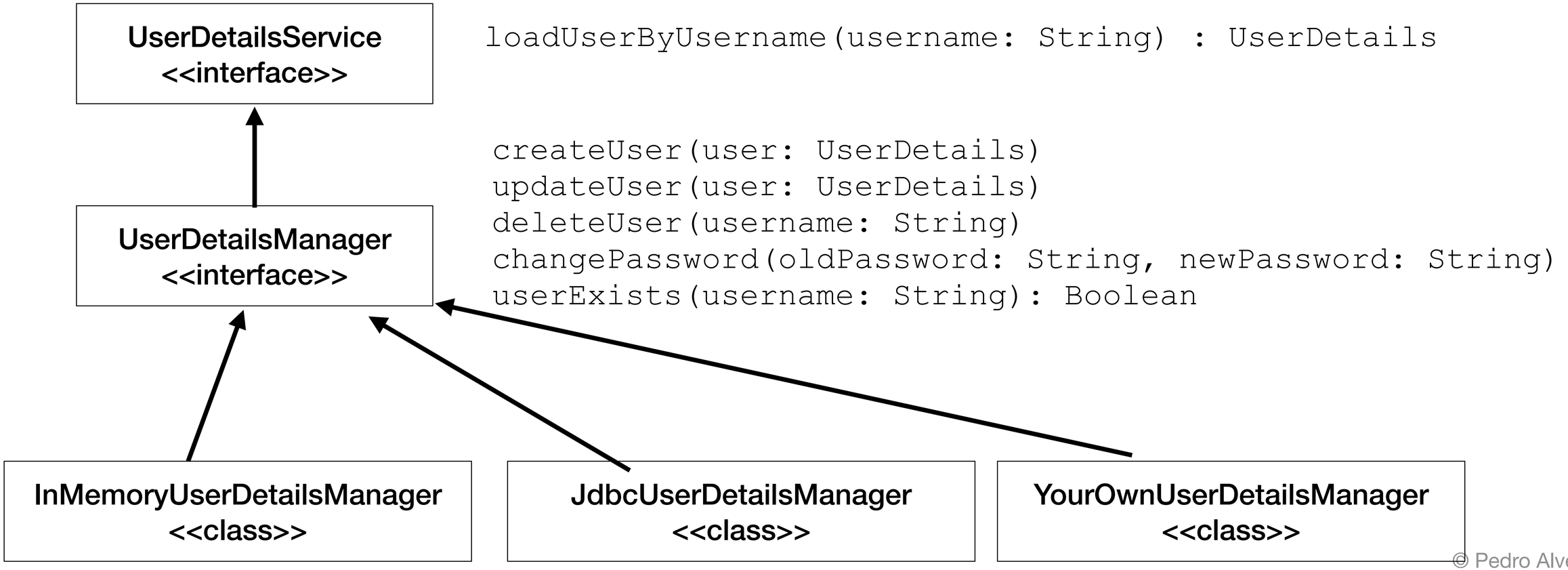
© Pedro Alves 2026

Spring Security

UserDetailsService

Revisions from week 5

UserDetailsService abstracts away how users are retrieved (memory, db, …)



© Pedro Alves 2026

Spring Security

UserDetailsService

Storing users in the database

- JdbcUserDetailsManager
  We must comply with Spring Security's expected schema/tables

- Custom UserDetailsManager (using JPA)
  More flexible; integrates with our own entities, repositories, and domain model

© Pedro Alves 2026

Spring Security

Custom UserDetailsManager

```
+----------+ 1      0..N +--------------+
| app_user |-------------| app_role     |
+----------+             +--------------+
| PK id    |             | PK id        |
| username |             | role (enum)  |
| password |             | FK user_id   |
+----------+             +--------------+
```

```
class AppUserDetailsManager(private val userRepository: AppUserRepository) : UserDetailsManager {

    override fun loadUserByUsername(username: String): UserDetails {
        val user: AppUser = userRepository.findByUsername(username) ?:
                                throw UsernameNotFoundException(username)
        return User(user.username, user.password,
                    user.roles.map { SimpleGrantedAuthority(it.role.name) })
    }

    ...

    override fun userExists(username: String): Boolean =
        userRepository.existsByUsername(username)
}
```

© Pedro Alves 2026

Spring Security

Custom UserDetailsManager

Registering our UserDetailsManager

```
@Configuration
@EnableWebSecurity
class SecurityConfig(private val appUserRepository: AppUserRepository) {

    @Bean
    fun passwordEncoder(): PasswordEncoder = BCryptPasswordEncoder()

    @Bean
    fun userDetailsManager(): UserDetailsManager = AppUserDetailsManager(appUserRepository)

}
```

© Pedro Alves 2026

Spring Security

DataInitializer

Creating users on startup (if they don't exist yet)

```
@Component
class DataInitializer(
    private val userDetailsManager: UserDetailsManager,
    private val passwordEncoder: PasswordEncoder
) : ApplicationRunner {

    override fun run(args: ApplicationArguments) {
        if (!userDetailsManager.userExists("alice")) {
            listOf(
                User("alice",   passwordEncoder.encode("password123"), listOf(SimpleGrantedAuthority("ROLE_EDITOR"))),
                User("bob",     passwordEncoder.encode("password123"), listOf(SimpleGrantedAuthority("ROLE_EDITOR"))),
                User("charlie", passwordEncoder.encode("password123"), listOf(SimpleGrantedAuthority("ROLE_ADMIN"))),
            ).forEach { userDetailsManager.createUser(it) }
        }
    }
}
```

© Pedro Alves 2026

Spring Security

Thymeleaf

Thymeleaf integrates with Spring Security using this dependency

```
<dependency>
    <groupId>org.thymeleaf.extras</groupId>
    <artifactId>thymeleaf-extras-springsecurity6</artifactId>
</dependency>
```

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
      xmlns:sec="http://www.thymeleaf.org/extras/spring-security">
```

```
<div sec:authorize="isAuthenticated()">
    Welcome back!
</div>

<div sec:authorize="hasRole('ADMIN')">
    <a th:href="@{/admin}">Admin panel</a>
</div>
```

```
<!-- fills in with authenticated username -->
<span sec:authentication="name"></span>
```

© Pedro Alves 2026

Exercise

	Basic Auth	Form-based login + session	Form-based login + JWT
State Management (stateful, stateless)			
Credentials usage (sent once, sent every request)			
Security exposure (low, medium, high)			
Scalability (high, low)			
User Experience (good, poor)			
Spring configuration (simple, complex)			
Login survives server restart (yes/no)			
Good for			

© Pedro Alves 2026

Resolution

	Basic Auth	Form-based login + session	Form-based login + JWT
State Management (stateful, stateless)	stateless	stateful	stateless
Credentials usage (sent once, sent every request)	sent every request	sent once	sent once
Security exposure (low, medium, high)	high	low	medium
Scalability (high, low)	high	low	high
User Experience (good, poor)	poor	good	good
Spring configuration (simple, complex)	simple	simple	complex
Login survives server restart (yes/no)	yes	no	yes
Good for	APIs, internal tools	Servers with low to moderate load	High load servers

© Pedro Alves 2026

Authentication in Internet Applications

- **HTTP Basic Authentication** - credentials are sent on every request
- **Form-based Authentication + Session** - sessionId is sent on every request (session data stored in the server)
- **Form-based Authentication + JWT** - session data is sent on every request
- **OAuth 2.0** - authentication is done in third-party service

© Pedro Alves 2026

OAuth 2.0

Provides a protocol for authorisation for Internet applications, resource owners, through third-parties on behalf of a principal.

OAuth defines four roles (from the RFC):

- **resource owner**: An entity capable of granting access to a protected resource. When the resource owner is a person, it is referred to as an end-user.
- **resource server**: The server hosting the protected resources, capable of accepting and responding to protected resource requests using access tokens.
- **client**: An application making protected resource requests on behalf of the resource owner and with its authorisation.
- **authorization server**: The server issuing access tokens to the client after successfully authenticating the resource owner and obtaining authorisation.

https://datatracker.ietf.org/doc/html/rfc6749

© Pedro Alves 2026

OAuth 2.0

Provides a protocol for **authorisation** for Internet applications, resource owners, through third-parties on behalf of a principal.

OAuth defines four roles (from the RFC):

- **resourc** When the reso
- **resourc** ng and respond
- **client**: rce owner and with its authorisation.
- **authorization server**: The server issuing access tokens to the client after successfully authenticating the resource owner and obtaining authorisation.

OAuth 2.0 is an authorization protocol, not an authentication protocol. However it is often used an authentication mechanism.

https://datatracker.ietf.org/doc/html/rfc6749

© Pedro Alves 2026

OAuth 2.0

OAuth 2.0 answers the question:

Can this app access this resource on behalf of the user?

Not:

Can this app get a proof of identity of the user?

© Pedro Alves 2026

OAuth 2.0 protocol flow

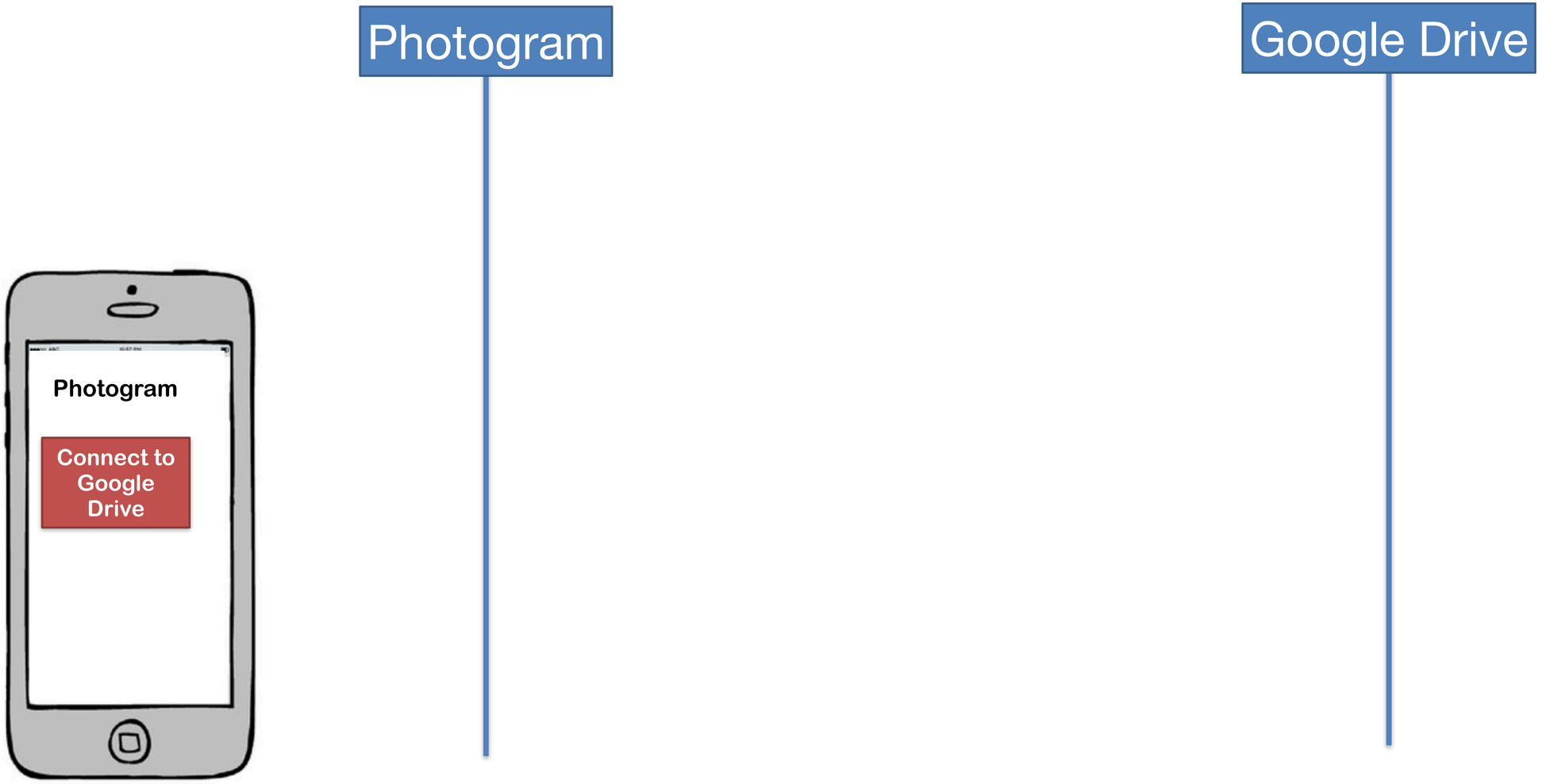
Example:

Photogram - An application to edit photos stored in google drive

Photogram wants to access Google drive files on behalf of the user

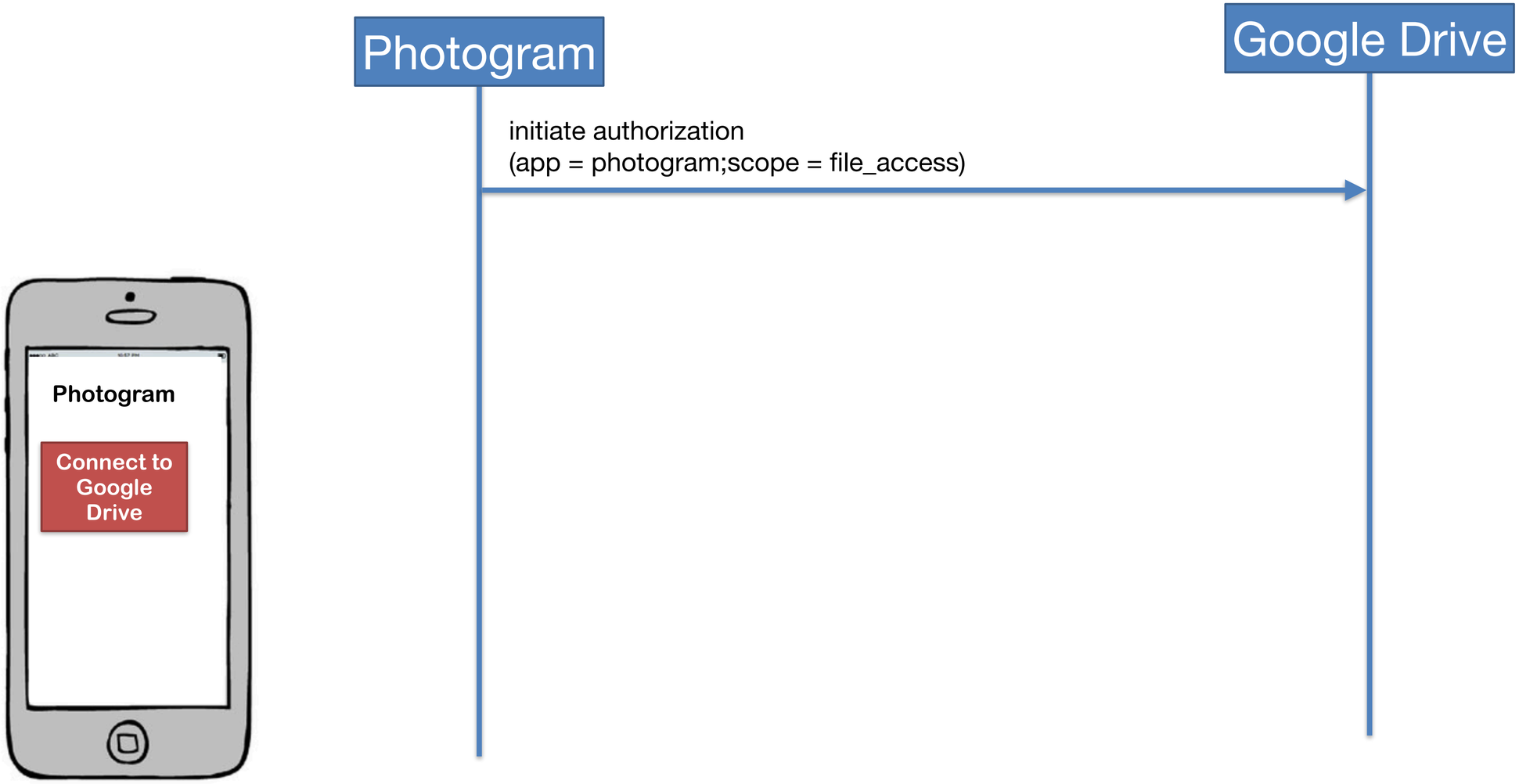
© Pedro Alves 2026

OAuth 2.0 protocol flow



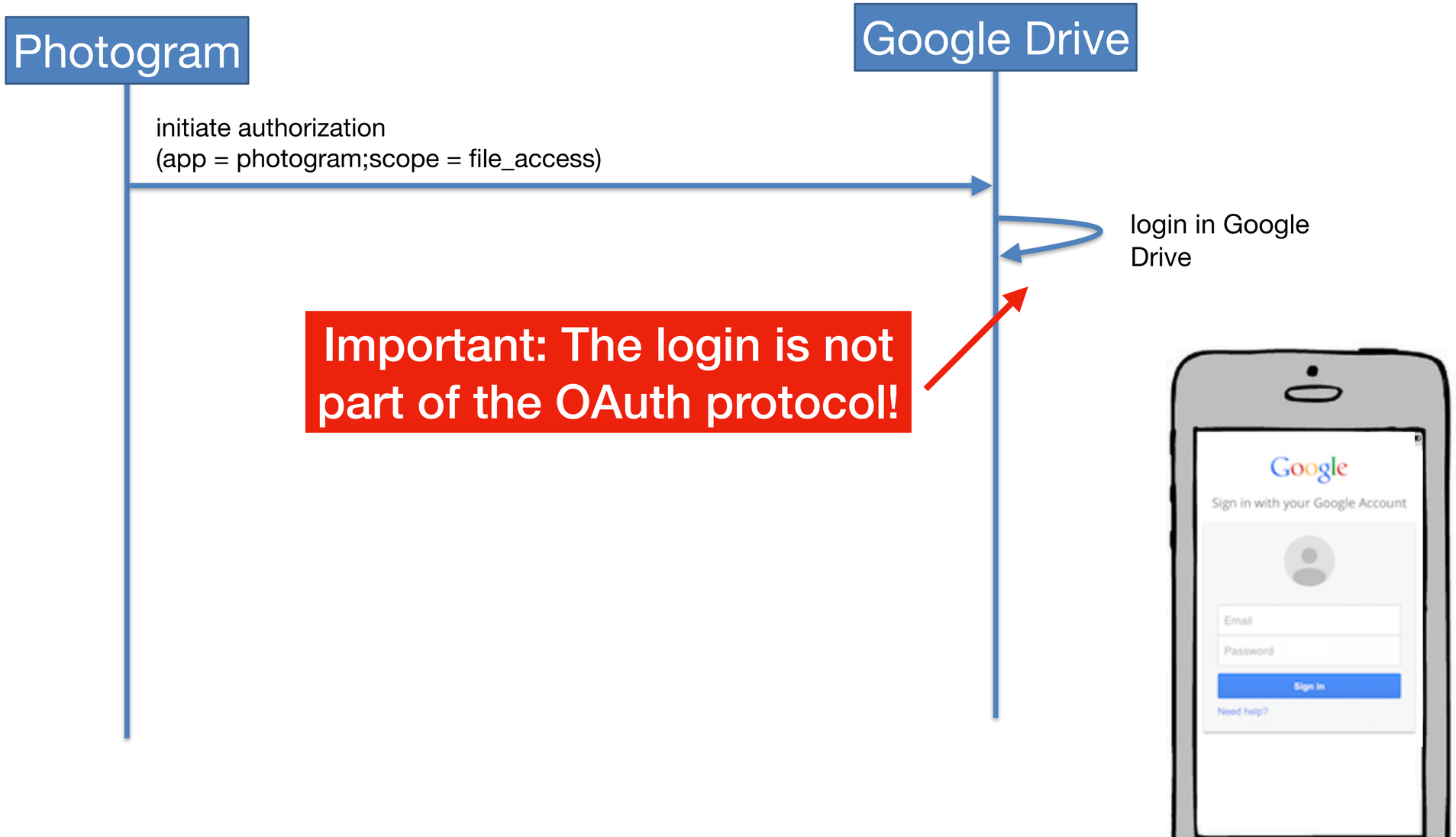
© Pedro Alves 2026

OAuth 2.0 protocol flow

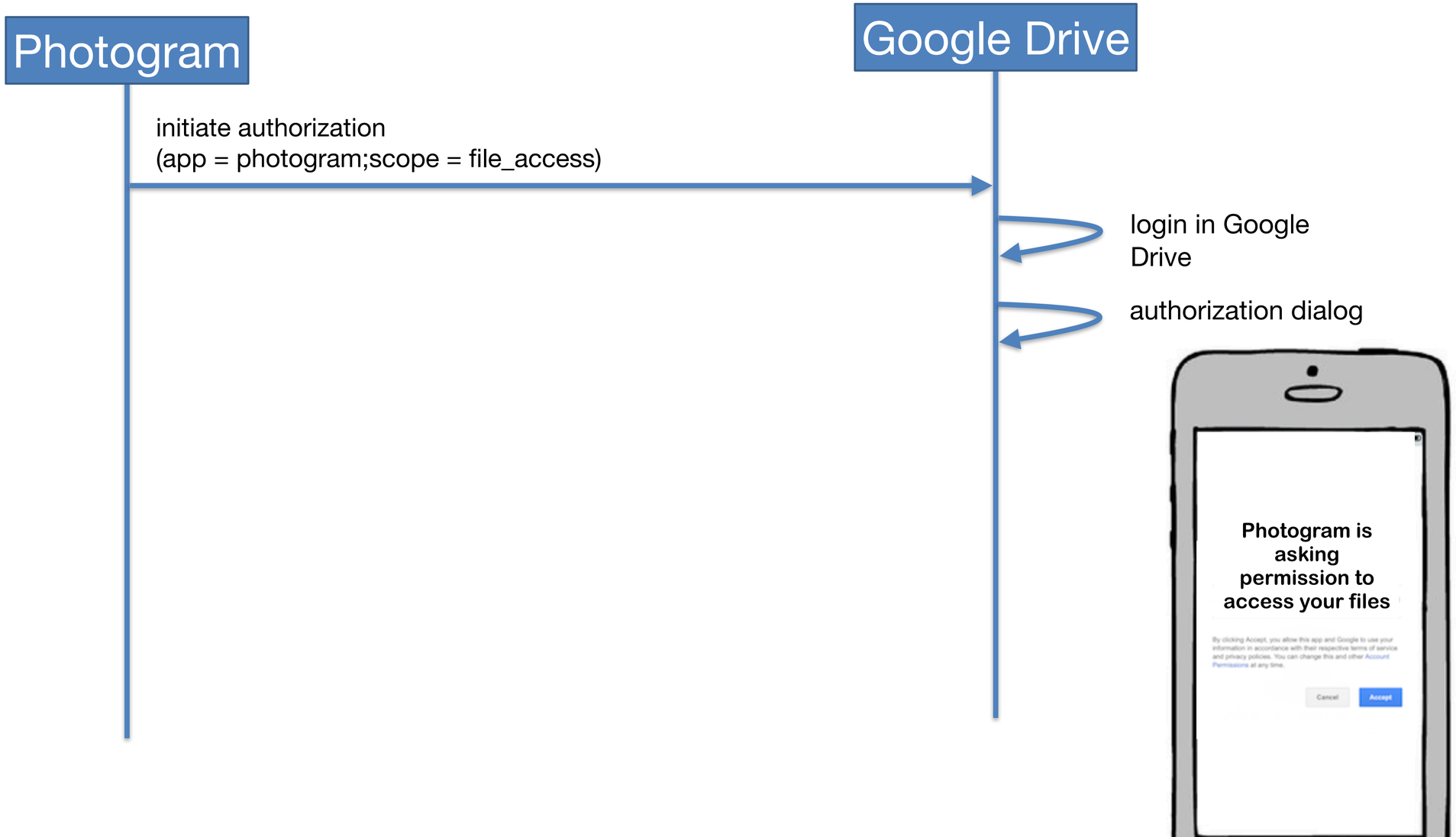


© Pedro Alves 2026

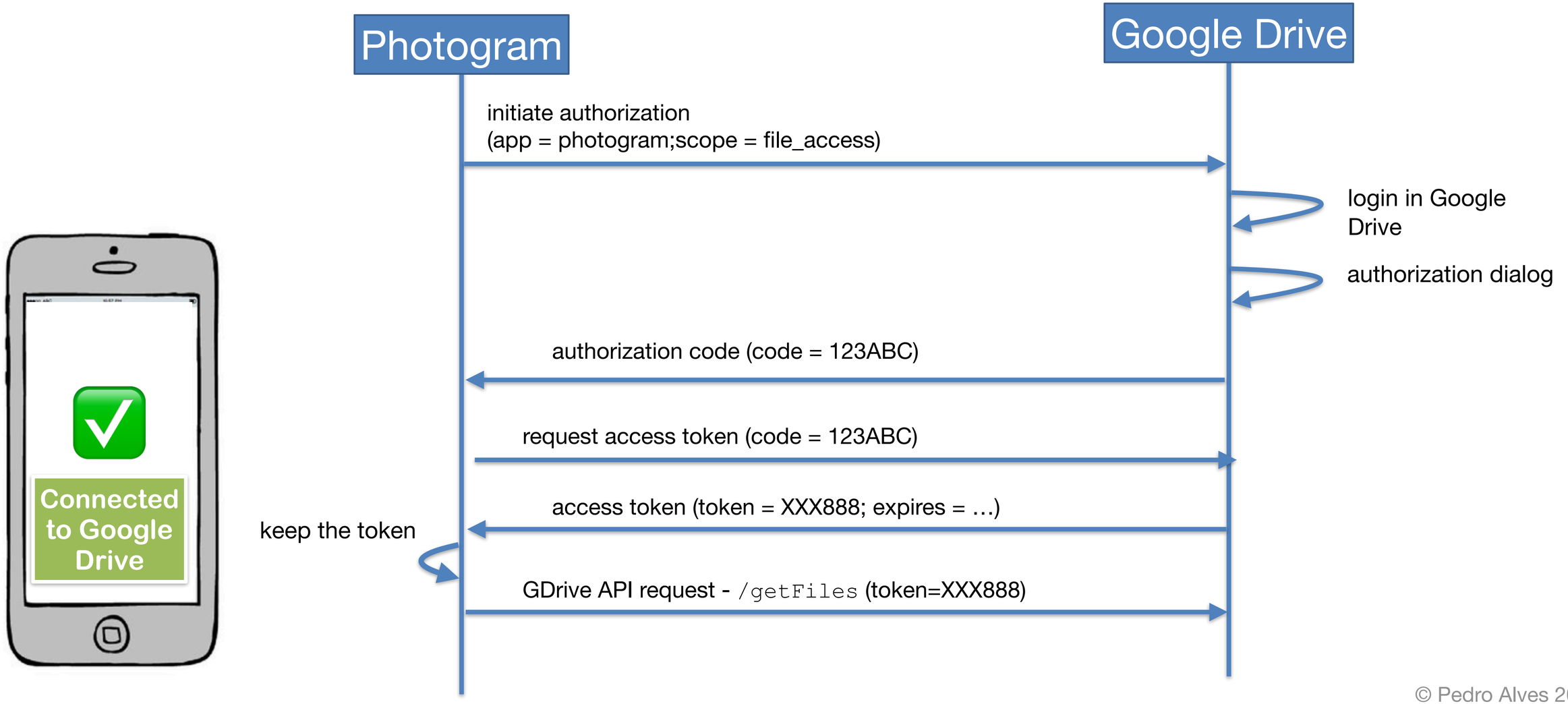
OAuth 2.0 protocol flow



OAuth 2.0 protocol flow



OAuth 2.0 protocol flow



© Pedro Alves 2026

OAuth 2.0 protocol flow

Photogram

Google Drive

initiate authorization
(app = photogram;scope = file_access)

login in Google Drive

authorization dialog

authorization code (code = 123ABC)

request access token (code = 123ABC)

access token (token = XXX888; expires = …)

keep the token

GDrive API request - `/getFiles` (token=XXX888)

`{"files": ["img1.png","img2.png"]}`

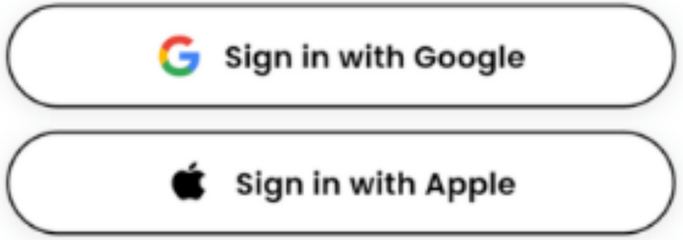
© Pedro Alves 2026

OAuth 2.0 protocol flow

```
+--------+                                +---------------+
|        |--(A)- Authorization Request ->|   Resource    |
|        |                               |     Owner     |
|        |<-(B)-- Authorization Grant ---|               |
|        |                               +---------------+
|        |
|        |                               +---------------+
|        |--(C)-- Authorization Grant -->| Authorization |
| Client |                               |     Server    |
|        |<-(D)----- Access Token -------|               |
|        |                               +---------------+
|        |
|        |                               +---------------+
|        |--(E)----- Access Token ------>|    Resource   |
|        |                               |     Server    |
|        |<-(F)--- Protected Resource ---|               |
+--------+                               +---------------+
```

© Pedro Alves 2026

OAuth 2.0 Login



These are usually called IdP (Identity Provider)

1. User is redirected to the provider
2. User logs in there
3. App receives an authorization code
4. App exchanges it for an access token

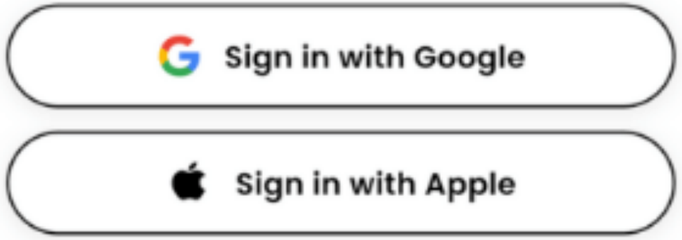
OAuth 2.0 protocol

5. App calls /userinfo
6. App gets user identity → logs user in

Get user identity

© Pedro Alves 2026

OAuth 2.0 Login



1. User is redirected to the provider
2. User logs in there
3. App receives an authorization code
4. App exchanges it for an access token
5. App calls /userinfo
6. App gets user identity → logs user in

Problem: This is not standard

- This endpoint may not exist in the provider
- The returned JSON doesn't have the same structure across providers

© Pedro Alves 2026

OIDC

OpenID Connect (OIDC)

OIDC is a thin layer on top of OAuth 2.0 that adds authentication.

The Access Token (JSON) now includes an Id Token (JWT signed by the Provider)

```
{
    "access_token": "ACCESS_TOKEN",
    "id_token": "JWT_HERE",
    "refresh_token": "REFRESH_TOKEN",
    "token_type": "Bearer",
    "expires_in": 3600
}
```

```
+--------+                                +---------------+
|        |--(A)- Authorization Request ->|   Resource    |
|        |                               |     Owner     |
|        |<-(B)-- Authorization Grant ---|               |
|        |                               +---------------+
|        |
|        |                               +---------------+
|        |--(C)-- Authorization Grant -->| Authorization |
| Client |                               |     Server    |
|        |<-(D)----- Access Token -------|               |
|        |                               +---------------+
|        |
|        |                               +---------------+
|        |--(E)----- Access Token ------>|    Resource   |
|        |                               |     Server    |
|        |<-(F)--- Protected Resource ---|               |
+--------+                               +---------------+
```

© Pedro Alves 2026

Authentication without OIDC

1. User is redirected to the provider
2. User logs in there
3. App receives an authorization code
4. App exchanges it for an access token
5. App calls /userinfo
6. App gets user identity → logs user in

As long as you know the UserInfo endpoint and the response format, you can authenticate your users

This is the case of GitHub

© Pedro Alves 2026

Spring

OAuth 2.0

Spring Boot provides an implementation for OAuth 2.0 that is easy to configure by loading a single module:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-oauth2-client</artifactId>
</dependency>
```

and configuring two properties (in this case, for github):

```
spring.security.oauth2.client.registration.github.client-id=client-id
spring.security.oauth2.client.registration.github.client-secret=client-secret
```

Get these from GitHub

© Pedro Alves 2026

Spring
OAuth 2.0

https://github.com/settings/developers > New OAuth app

Register a new OAuth app

Application name *

My App

Something users will recognize and trust.

Homepage URL *

http://localhost:8080

The full URL to your application homepage.

Application description

Application description is optional

This is displayed to all users of your application.

Authorization callback URL *

http://localhost:8080/login/oauth2/code/github

Your application's callback URL. Read our OAuth documentation for more information.

☐ **Enable Device Flow**

Allow this OAuth App to authorize users via the Device Flow.

Read the Device Flow documentation for more information.

Register application Cancel

My App

palves-ulht owns this application. Transfer ownership

You can list your application in the GitHub Marketplace so that other users can discover it. List this application in the Marketplace

0 users Revoke all user tokens

Client ID

0v23li3kEJ8qkBymbN8P

Client secrets Generate a new client secret

Make sure to copy your new client secret now. You won't be able to see it again.

Client secret

✓ 588417435a[redacted]

Added now by **palves-ulht**

Never used

You cannot delete the only client secret. Generate a new client secret first.

Delete

© Pedro Alves 2026

Spring

OAuth 2.0

```
@Bean
fun securityFilterChain(http: HttpSecurity): SecurityFilterChain {
    http
        .authorizeHttpRequests { auth -> ... }
        .oauth2Login { }
    ...
}
```

Enable OAuth 2.0 login

More info: https://docs.spring.io/spring-security/reference/servlet/oauth2/login/core.html

© Pedro Alves 2026

Spring

OAuth 2.0 - Customizing user name

After logging in via OAuth 2.0, what will the principal name be?

1. Login Protocol via OAuth 2.0, ends with an access token
2. Spring will automatically call the IdP default userInfo endpoint, using the access token
   Example for GitHub: https://api.github.com/user
3. The endpoint returns a json with user info. Spring tries to find the attribute that represents the subject identity (for GitHub it will be “id”)
   It can be overriden by `spring.security.oauth2.client.provider.github.user-name-attribute=login`
4. Spring creates an Authentication object with that id and adds it to the SecurityContext

If we don’t set the user-name-attribute, the principal name will be the github user id (e.g. 34435467)

© Pedro Alves 2026

Spring

OAuth 2.0 - Customizing user info

The userInfo endpoint may include other relevant information (full name, avatar, email, location, etc…) that we want to associate with the (app) user

Create a class that implements OAuth2UserService

```
@Service
class CustomOAuth2UserService(
    private val appUserRepository: AppUserRepository
) : OAuth2UserService<OAuth2UserRequest, OAuth2User> {

    private val delegate = DefaultOAuth2UserService()

    override fun loadUser(userRequest: OAuth2UserRequest): OAuth2User {
        val oauth2User = delegate.loadUser(userRequest)
        val githubLogin = oauth2User.getAttribute<String>("login")!!

        val avatarUrl = oauth2User.getAttribute<String>("avatar_url")

        // create or update the user in the database

        val authorities = listOf(SimpleGrantedAuthority(Role.ROLE_NONE.name))
        return DefaultOAuth2User(authorities, oauth2User.attributes, "login")
    }
}
```

Issues a request to the userInfo endpoint

This class extends AuthenticationPrincipal and will be added to the SecurityContext

© Pedro Alves 2026

Spring

OAuth 2.0 - Customizing user info

Inject that class (bean) into the oAuth pipeline

SecurityConfig

```
@Bean
fun securityFilterChain(http: HttpSecurity): SecurityFilterChain {
    http
        .authorizeHttpRequests { auth -> ... }
        .oauth2Login {
           oauth2.userInfoEndpoint {
              it.userService(customOAuth2UserService)
           }
        }
    ...
}
```

© Pedro Alves 2026

OAuth 2.0

What about roles?

Option 1 - The IdP only proves identity, your app manages roles

Option 2 - The IdP knows user groups/ roles and provides them through the userInfo endpoint

© Pedro Alves 2026

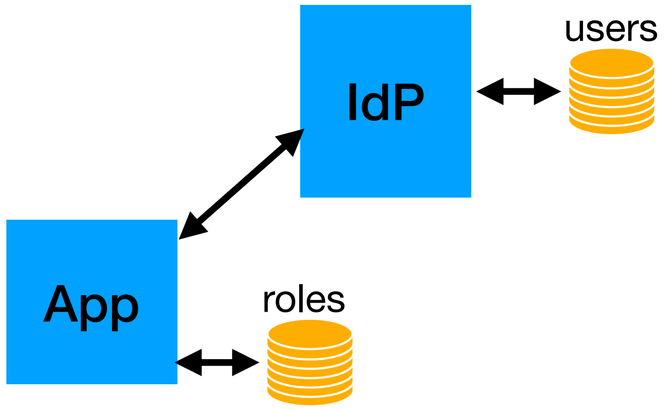
OAuth 2.0

What about roles?

Option 1 - The IdP only proves identity, your app manages roles

Option 2 - The IdP knows user groups/ roles and provides them through the userInfo endpoint

user_id	user_name
1345	palves
2354	darthvader



user_id	idp_user_name	user_role
1	palves	ADMIN
2	darthvader	EDITOR

© Pedro Alves 2026

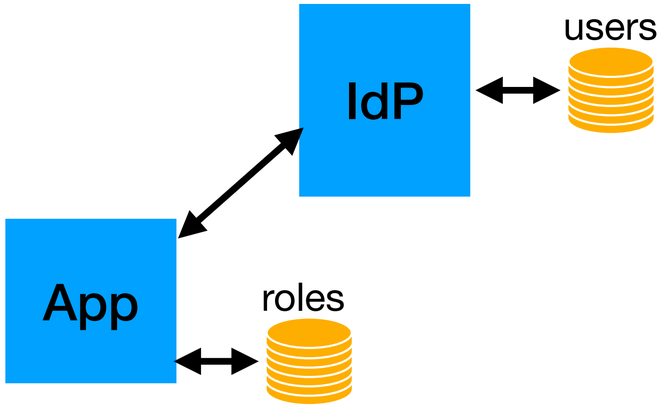
Spring

OAuth 2.0 - App Role Management

Use the class that implements OAuth2UserService to get the roles from the DB

```
@Service
class CustomOAuth2UserService(
    private val appUserRepository: AppUserRepository
) : OAuth2UserService<OAuth2UserRequest, OAuth2User> {

    private val delegate = DefaultOAuth2UserService()

    override fun loadUser(userRequest: OAuth2UserRequest): OAuth2User {
        val oauth2User = delegate.loadUser(userRequest)
        val githubUsername = oauth2User.getAttribute<String>("login")!!

        val appUser = appUserRepository.findByIdpUsername(githubUsername)
                      ?: provisionUser(githubUsername)

        val authorities = appUser.roles.map { SimpleGrantedAuthority(it.role.name) }

        return DefaultOAuth2User(authorities, oauth2User.attributes, "login")
    }

    // Auto-provisions a local user record on first GitHub login with a default role.
    // An admin can later update the role by changing the AppUser in the database.
    private fun provisionUser(username: String): AppUser {
        ...
    }
}
```

user_id	user_name
1345	palves
2354	darthvader



user_id	idp_user_name	user_role
1	palves	ADMIN
2	darthvader	EDITOR

© Pedro Alves 2026

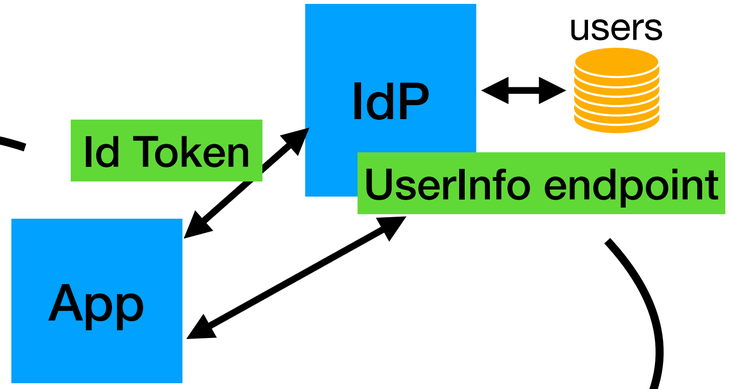
OAuth 2.0

What about roles?

Option 1 - The IdP only proves identity, your app manages roles

Option 2 - The IdP knows user groups/roles and provides them either throw claims in the ID Token (OIDC) or through the userInfo endpoint

user_id	user_name	user_role
1345	palves	ADMIN
2354	darthvader	EDITOR




```
{
    "sub": "palves",
    "exp": 1710000000,
    "iat": 1709996400,

    "email": “pedro@mail.com",
    "name": “Pedro Alves",

    "roles": ["USER", "ADMIN"]
}
```

```
{
    "sub": "palves",
    "name": “Pedro Alves",
    "email": “pedro@mail.com”,
    “roles": ["ADMIN", "USER"]
}
```

© Pedro Alves 2026

Exercise

	OAuth 2.0 (pure)	OAuth 2.0 + UserInfo endpoint	OIDC
Authentication (yes/no)			
Authorization (yes/no)			
Token Type (access token, id token)			
User Information (none, standard, custom)			
Allows roles (standard, custom,no)			

© Pedro Alves 2026

Solution

	OAuth 2.0 (pure)	OAuth 2.0 + UserInfo endpoint	OIDC
Authentication (yes/no)	no	yes	yes
Authorization (yes/no)	yes	no	yes
Token Type (access token, id token)	access token	access token	access token + id token
User Information (none, standard, custom)	none	custom	standard
Allows roles (standard, custom,no)	no	custom (through UserInfo endpoint)	custom (through id token)

© Pedro Alves 2026